Lab 5 — PKI: Build Your Own Certificate Authority

You will stand up a private root CA, issue a server certificate with a CSR, and revoke it. This covers Security+ 1.4: CA, CSR, root of trust, CRL, self-signed vs. third-party.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free): - `openssl` (pre-installed)

Step 1 — Create the CA directory layout

```
mkdir -p ~/ca/{certs,private,newcerts,crl}
```

```
cd ~/ca
```

```
touch index.txt
```

```
echo 1000 > serial
```

```
echo 1000 > crlnumber
```

Step 2 — Generate the root CA key + self-signed root certificate

```
openssl genrsa -out private/ca.key 4096
```

```
openssl req -x509 -new -key private/ca.key -days 3650 -sha256 \
```

```
    -subj "/CN=Lab Root CA/O=Tertiary InfoTech" \
```

```
    -out certs/ca.crt
```

```
openssl x509 -in certs/ca.crt -noout -subject -issuer
```

Subject == Issuer means it is self-signed. This is your **root of trust**.

Step 3 — Generate a server key + CSR

```
openssl genrsa -out private/server.key 2048
```

```
openssl req -new -key private/server.key \
```

```
    -subj "/CN=web.lab.local/O=Lab" \
```

```
    -out server.csr

openssl req -in server.csr -noout -subject
```

The CSR contains the public key + identity, but no signature from a CA yet.

Step 4 — Sign the CSR with your CA

```
cat > openssl.cnf <<'EOF'

[ ca ]

default_ca = CA_default

[ CA_default ]

dir = .

database = $dir/index.txt

serial = $dir/serial
```

```
new_certs_dir = $dir/newcerts

certificate = $dir/certs/ca.crt

private_key = $dir/private/ca.key

crlnumber = $dir/crlnumber

default_md = sha256

policy = pol

default_days = 365

default_crl_days = 30

[ pol ]

commonName = supplied
```

```
organizationName = optional

EOF

openssl ca -batch -config openssl.cnf -in server.csr -out certs/server.crt

openssl x509 -in certs/server.crt -noout -issuer -subject
```

Subject == `web.lab.local`, Issuer == `Lab Root CA`. The chain works.

Step 5 — Revoke and publish a CRL

```
openssl ca -config openssl.cnf -revoke certs/server.crt

openssl ca -config openssl.cnf -gencrl -out crl/lab.crl

openssl crl -in crl/lab.crl -noout -text | head -15
```

The CRL now lists `server.crt` as revoked. A relying party would refuse to trust it.

Step 6 — Compare with third-party CAs

Browse the trusted-root store on this machine:

```
ls /etc/ssl/certs | head
```

```
awk '/Subject:/ {print}' /etc/ssl/certs/ca-certificates.crt | sort -u
| head
```

Real public sites get certificates from these third-party CAs (Let's Encrypt, DigiCert, …) instead of a self-signed root.

Free online tools

- **Let's Encrypt** — free public CA: https://letsencrypt.org
- **SSL Labs Server Test** — https://www.ssllabs.com/ssltest
- **crt.sh** — public certificate transparency search: https://crt.sh
- **X509 decoder** — https://www.sslshopper.com/certificate-decoder.html

---

What you learned

- The exact files a CA needs (`index.txt`, `serial`, key, root cert).
- The lifecycle: keypair → CSR → CA-signed cert → revocation → CRL.
- Why public trust requires a third-party CA, not a self-signed root.